

# PHASE 11 SOP

## Kubernetes StatefulSets, Headless Services & Persistent Storage

### Phase Overview

#### What Are We Going To Learn?

In this phase we move beyond stateless applications and learn how Kubernetes runs stateful workloads such as databases.

Examples:

- MySQL
- PostgreSQL
- MongoDB
- Apache Kafka

These applications require:

Stable Identity

Stable Networking

Persistent Storage

which Deployments cannot guarantee.

#### Why Do We Need StatefulSets?

Deployments are designed for stateless applications.

Example:

Nginx

Frontend Applications

REST APIs

When a Deployment pod dies:

nginx-abc123

↓

nginx-xyz789

A completely new pod is created.

This is acceptable for stateless applications.

However databases require:

db-0

db-1

db-2

to maintain their identity.

If identities keep changing:

Database Replication

Cluster Communication

Leader Election

can break where StatefulSets solve this problem.

## Phase 11A

### Understanding StatefulSet Identity

#### Objective

Understand how StatefulSets provide stable pod identities.

#### Create StatefulSet

File:

```
apiVersion: apps/v1
```

```
kind: StatefulSet
```

```
metadata:
```

```
  name: web
```

```
spec:
```

```
  serviceName: "web"
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx
```

```
template:
  metadata:
    labels:
      app: nginx

  spec:
    containers:
      - name: nginx
        image: nginx:latest
```

Apply:

```
kubectl apply -f statefulset.yaml
```

[Verify StatefulSet:](#)

```
kubectl get statefulset
```

Expected:

| NAME | READY |
|------|-------|
| web  | 3/3   |

[Verify Pods:](#) Kubectl get pods

Observe:

Deployment Pods:  
nginx-random-name

StatefulSet Pods:  
web-0  
web-1  
web-2

[Prove Stable Identity](#)

Delete a pod:

```
kubectl delete pod web-1
```

Watch:

```
kubectl get pods -w
```

Observe:

web-1

dies

↓

web-1

returns

Identity remains unchanged.

## Phase 11B

### Headless Services

#### Objective

Provide stable networking identities for StatefulSet pods.

#### Why?

Pods communicate using DNS names rather than IP addresses.

IPs may change.

DNS names remain constant.

#### Create Headless Service

File:

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: web
```

```
spec:
```

```
  clusterIP: None
```

```
  selector:
```

```
    app: nginx
```

```
  ports:
```

```
    - port: 80
```

Apply:

```
kubectl apply -f headless-service.yaml
```

Verify

```
kubectl get svc
```

Expected:

```
web      ClusterIP      None
```

Important:

ClusterIP = None

means:

Headless Service

## Phase 11C

### Stable DNS Records

Objective

Prove StatefulSet pods receive permanent DNS identities.

Create Temporary DNS Test Pod

```
kubectl run dns-test `
  --image=busybox:1.35 `
  --restart=Never `
  -it -- sh
```

Verify DNS Records

Inside BusyBox:

```
nslookup web-0.web.default.svc.cluster.local
```

Example:

Name:

```
web-0.web.default.svc.cluster.local
```

Address:

```
172.31.x.x
```

Repeat:

```
nslookup web-1.web.default.svc.cluster.local
```

```
nslookup web-2.web.default.svc.cluster.local
```

### Key Learning

DNS records are created automatically:

```
web-0.web.default.svc.cluster.local
```

```
web-1.web.default.svc.cluster.local
```

```
web-2.web.default.svc.cluster.local
```

Applications communicate using names rather than changing IPs.

## Phase 11D

### StatefulSet Persistent Storage

#### Objective

Provide dedicated persistent storage for every StatefulSet pod.

#### Problem

Current StatefulSet:

Identity 

DNS 

Storage 

Verify: `kubectl describe statefulset web`

Output: Volume Claims: <none>

### StatefulSet With Storage

File:

```
apiVersion: apps/v1
```

```
kind: StatefulSet
```

```
metadata:
```

```
  name: web
```

```
spec:
  serviceName: "web"

  replicas: 3

  selector:
    matchLabels:
      app: nginx

  template:
    metadata:
      labels:
        app: nginx

    spec:
      containers:
        - name: nginx
          image: nginx:latest

          volumeMounts:
            - name: data
              mountPath: /usr/share/nginx/html

  volumeClaimTemplates:
    - metadata:
        name: data

      spec:
        accessModes:
          - ReadWriteOnce

        storageClassName: ebs-sc

        resources:
          requests:
            storage: 1Gi
```

Apply:

```
kubectl apply -f statefulset-storage.yaml
```

## Verify PVC Creation

```
kubectl get pvc
```

Expected:

```
data-web-0    Bound
data-web-1    Bound
data-web-2    Bound
```

**What Happened?** Kubernetes automatically created:

web-0

↓

data-web-0

↓

EBS Volume

web-1

↓

data-web-1

↓

EBS Volume

web-2

↓

data-web-2

↓

EBS Volume

## Phase 11E

### Proving Storage Persistence

#### Objective

Prove data survives pod recreation.

#### Create Test File

Enter pod:

```
kubectl exec -it web-0 -- sh
```

Create file:

```
echo "hello from web-0" > /usr/share/nginx/html/test.txt
```

Verify:

```
cat /usr/share/nginx/html/test.txt
```

Output:

```
hello from web-0
```

Exit:

```
exit
```

Delete Pod

```
kubectl delete pod web-0
```

Watch:

```
kubectl get pods -w
```

Wait until:

```
web-0    Running
```

returns.

Verify Data

```
Reconnect: kubectl exec -it web-0 -- sh
```

```
Check file: cat /usr/share/nginx/html/test.txt
```

Output: hello from web-0

Key Learning

Pod was destroyed: web-0 ❌

Storage survived:

data-web-0 PVC ✅

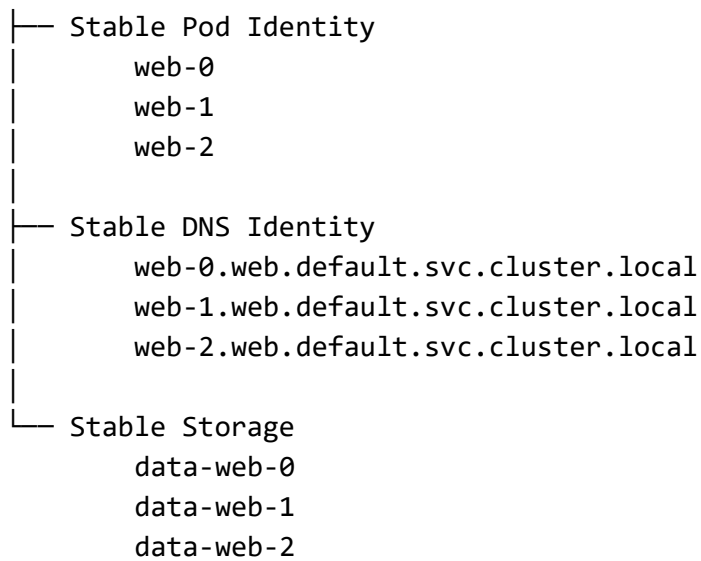
EBS Volume ✅

New pod was created: web-0 ✅ & got reattached to the same volume.

Production Architecture Summary

StatefulSet

|



### Interview Summary: When should StatefulSets be used?

Use StatefulSets when applications require:

Stable Pod Names

Stable Network Identities

Persistent Storage

Examples:

- MySQL
- PostgreSQL
- MongoDB
- Apache Kafka

Deployments are preferred for stateless workloads, while StatefulSets are designed for stateful distributed systems.

## Appendix A - Troubleshooting & Lessons Learned

### Issue 1 - StatefulSet DNS Resolution Failed

#### Symptom

Inside BusyBox:

```
nslookup web-0.web
```

Output:

```
NXDOMAIN
```

## Root Cause

We initially tested using a short DNS name.

Kubernetes did not resolve:

`web-0.web`

in our test environment.

## Resolution

Use the fully qualified domain name:

`nslookup web-0.web.default.svc.cluster.local`

Example Output:

Name: `web-0.web.default.svc.cluster.local`

Address: `172.31.41.162`

## Learning

StatefulSet networking relies on:

Pod Name

+

Headless Service

+

Namespace

forming:

`web-0.web.default.svc.cluster.local`

## Issue 2 - BusyBox DNS Test Pod Failed

### Symptom

`kubectl get pod dns-test`

Output:

STATUS: Error

### Root Cause

The BusyBox pod was launched interactively.

After exiting the shell:

```
exit
```

the container terminated.

### Resolution

Delete and recreate:

```
kubectl delete pod dns-test
```

```
kubectl run dns-test `
  --image=busybox:1.35 `
  --restart=Never `
  -it -- sh
```

### Learning

Interactive debug containers are expected to terminate when the shell exits.

This is normal behavior.

## Issue 3 - PVC Stuck In Pending State

### Symptom

```
kubectl get pvc
```

Output:

```
data-web-0    Pending
```

### Investigation

```
kubectl describe pvc data-web-0
```

Events: FailedBinding

no persistent volumes available for this claim  
and no storage class is set

### Root Cause

StatefulSet PVC template did not specify:

storageClassName:

Kubernetes had no instructions for provisioning storage.

### Resolution

Update:

`volumeClaimTemplates:`

to include:

`storageClassName: ebs-sc`

### Learning

PVCs do not automatically know where storage comes from.

A StorageClass must tell Kubernetes:

Use EBS

Use EFS

Use NFS

etc.

## Issue 4 - StatefulSet Update Rejected

**Symptom:** `kubectl apply -f statefulset-storage.yaml`

Output:

Forbidden:

updates to statefulset spec are forbidden

### Root Cause

We attempted to modify:

`volumeClaimTemplates`

after StatefulSet creation.

### Resolution

Delete StatefulSet:

`kubectl delete statefulset web`

Recreate using updated YAML.

### Learning

StatefulSet storage definitions are immutable.

Kubernetes protects storage configurations to prevent accidental data loss.

### **Issue 5 - StatefulSet Deleted But PVC Still Exists**

#### Symptom

```
kubectl delete statefulset web
```

Followed by:

```
kubectl get pvc
```

Output:

```
data-web-0    Pending
```

PVC remained.

#### Root Cause

StatefulSets do not automatically delete PVCs.

#### Resolution

Delete manually:

```
kubectl delete pvc data-web-0
```

#### Learning

This behavior protects production data.

Deleting:

StatefulSet

does NOT mean:

Delete Database Data

### **Issue 6 - Only data-web-0 Created Initially**

#### Symptom

Immediately after deployment:

```
kubectl get pvc
```

showed:

```
data-web-0
```

but not:

data-web-1

data-web-2

### Root Cause

StatefulSets create resources sequentially.

web-0

↓

web-1

↓

web-2

### Resolution

Wait for:

web-0

PVC to bind successfully.

Then Kubernetes proceeds to create:

web-1

and:

web-2

### Learning

Unlike Deployments, StatefulSets do not create all replicas simultaneously.

## Issue 7 - Verifying Storage Persistence

### Validation Performed

Created:

```
echo "hello from web-0" > test.txt
```

inside:

web-0

Deleted:

```
kubect1 delete pod web-0
```

Reconnected:

```
kubectl exec -it web-0 -- sh
```

Verified:

```
cat test.txt
```

Output:

```
hello from web-0
```

### Learning

Pod was destroyed.

Storage was not.

web-0

↓

data-web-0

↓

EBS Volume

The recreated pod automatically reattached the same storage.